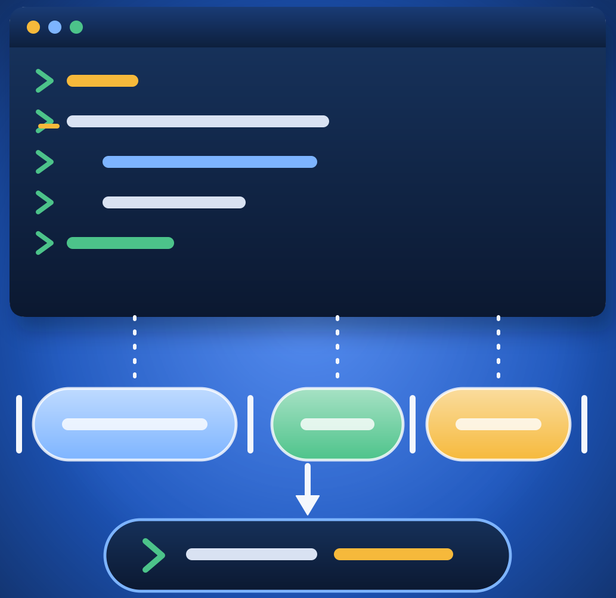
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 7

Bash Scripting and Safe Arguments

Lab 7 guide: turn commands into a script, keep every argument whole, and report failure with an exit status.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create and run a Bash script with arguments you understand and the right execute permission.
2. Handle spaces, names that start with a dash, and missing input without unwanted word splitting.
3. Test a reusable command, and explain its exit status and its safe handling of file names.

THIS LAB

Lab 7

First the concepts, with examples. Then the lab on the server. A personal PATH, login messages and the outbox are optional extensions, so they come last.

DISCUSSION

A file is called one file.txt. How many arguments is that?

To you it is one name. To the shell, with a space inside it, how many words is it?

CONCEPTS FIRST

Scripts, arguments and quoting

Four ideas: a script is a saved command, arguments need quotes, option parsing needs --, and every command ends with a status.

A script is a saved list of commands

show_args.sh

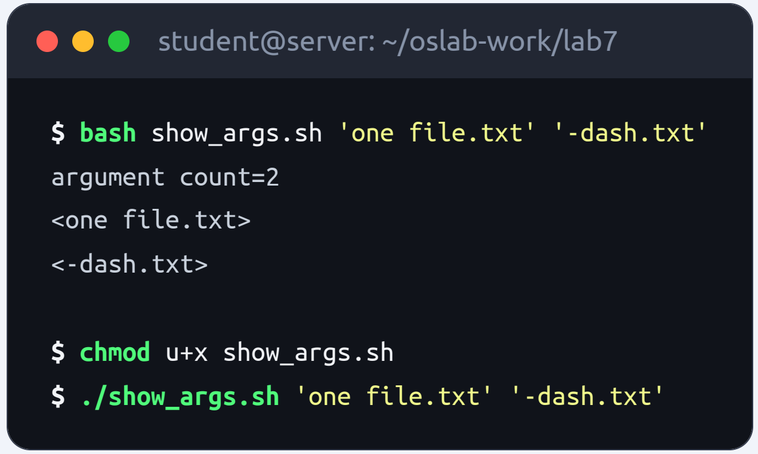
```
#!/usr/bin/env bash
printf 'argument count=%s\n' "$#"
for argument in "$@"; do
    printf '<%s>\n' "$argument"
done
```

The first line, the shebang, names the interpreter that should run the file.

$# is the number of arguments. $@ is the list of arguments. for repeats the body once for each of them.

printf with angle brackets shows exactly where each argument starts and ends.

Two ways to run it



bash script runs the file with Bash, whether or not it is executable.

./script runs it directly. That needs the execute bit and the shebang. ./ names the file in the current folder, which is not on PATH.

chmod u+x gives the owner the execute bit.

If a direct run says Permission denied, the file may lack the execute bit, or the folder may be mounted noexec. bash script is the allowed fallback.

Quotes keep an argument whole: "$@"

Quoted: "$@"

student@server: ~/oslab-work/lab7

```
$ for a in "$@"; do printf '<%s>\n' "$a"; done
<one file.txt>
```

Each argument stays one item, spaces inside it and all.

Unquoted: $@

student@server: ~/oslab-work/lab7

```
$ for a in $@; do printf '<%s>\n' "$a"; done
<one
<file.txt>
```

The shell splits on spaces. One name becomes two words, and the script looks for files that do not exist.

Quote every expansion that holds a file name: "$path", "$@", "$file".

A name that starts with a dash



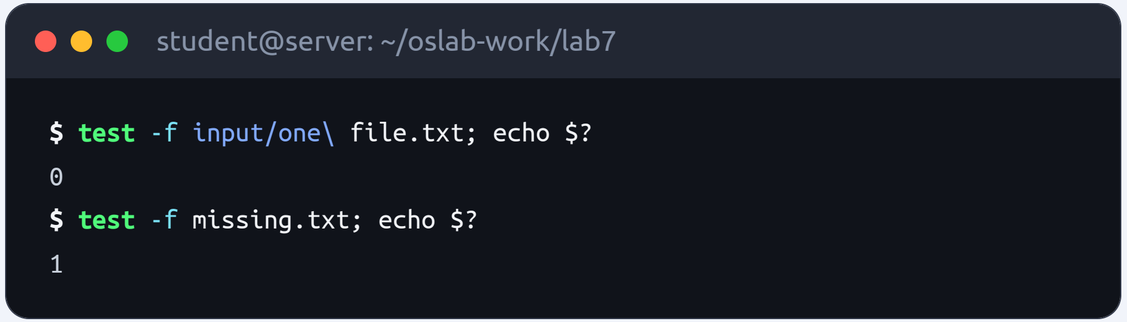
A word that starts with - looks like an option, so wc tries to read it as one.

-- tells a command that the options have ended. Everything after it is a file name.

Quoting alone does not help: the quotes are removed by the shell before wc sees the word. Writing ./-dash.txt also works.

Every command ends with an exit status

Status	Meaning
0	Success
1 and above	Some kind of failure
2	By habit, a wrong use of the command



$? holds the status of the last command. A script ends with the status you give to exit, or with the status of its last command.

LAB 7

Your own file command

Build a personal TechCorp command that inspects files. It must keep every argument whole.

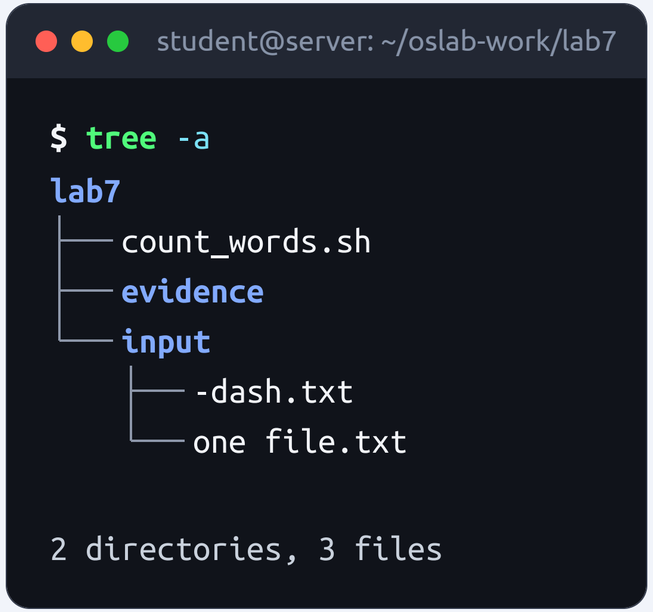
The scenario

SCENARIO

A command for your own files

Build a personal TechCorp command that inspects files in your own server neighbourhood. It must keep argument boundaries intact before any cross-user automation is considered.

The core stays in your own folders. Cross-user work needs a narrow folder the instructor prepared.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Task 1, guided: script basics
25 to 35	Written prediction
35 to 70	Task 2: your personal file command

Minutes	What happens
70 to 85	Normal, edge and leading-dash tests
85 to 100	Individual changed-case checkpoint
100 to 110	Correction and explanation
110 to 120	Cleanup and submission

How you work

Your own account

Log in with your own account. Use only your own files and processes. Do not use eval, and do not run scripts you did not write.

AI tools

Optional in the investigation and the tests. Not allowed in the prediction and the individual checkpoint.

Evidence

Keep what you observe in the evidence folder. Short, attributable evidence earns more than long output.

Initial prediction accuracy is lightly weighted. A reasoned attempt and an evidence-based correction earn credit.

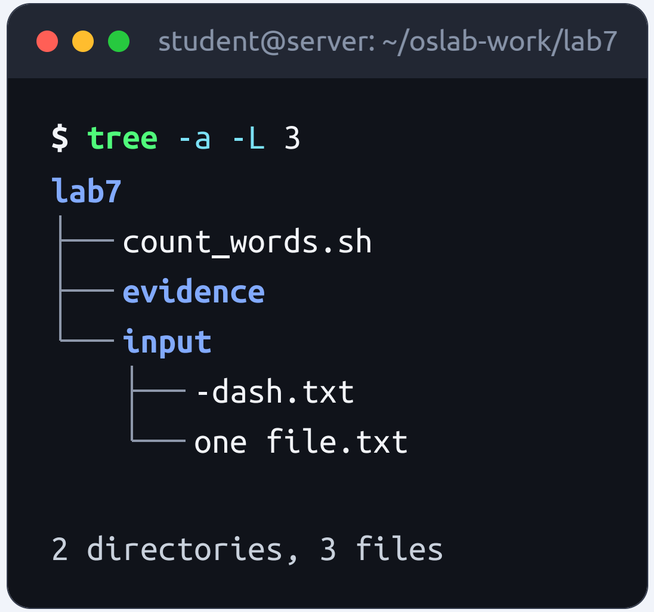
Setup: start the lab

student@server: ~

```
$ whoami
$ command -v oslab
$ oslab doctor
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab7
$ cd "$OSLAB_WORKSPACE/lab7"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- whoami shows your account. command -v oslab and oslab doctor check the helper.
- If the helper is missing, follow the local setup guide in the course repository.
- oslab start lab7 makes the lab folder. It keeps your work if you start again, and it does not change your current folder, so cd into it yourself.
- If you are resuming, look at the existing files before you run a command that writes to them.

Compare with the starting tree



count_words.sh is an editable starter skeleton.

input/ holds two files: one file.txt has two words, and -dash.txt has one.

The helper also writes a hidden file, .oslab-managed.json. Leave it as it is.

GUIDED

Script basics

Task 1, with the instructor: fifteen minutes.

Step 1: write the argument viewer

student@server: ~/oslab-work/lab7

```
$ cat > show_args.sh <<'SH'
$ #!/usr/bin/env bash
$ printf 'argument count=%s\n' "$#"
$ for argument in "$@"; do
$     printf '<%s>\n' "$argument"
$ done
$ SH
$ bash show_args.sh 'one file.txt' '-dash.txt'
```

cat > file <<'SH' writes everything up to the line SH into the file. This is a here-document.

The quotes around SH stop your own shell from expanding $# and $@ while you type. They are left for the script.

Expected: a count of 2, then two arguments in angle brackets.

Step 2: run it directly

student@server: ~/oslab-work/lab7

```
$ chmod u+x show_args.sh
$ ./show_args.sh 'one file.txt' '-dash.txt'
```

The shebang picks the interpreter for a direct run. ./ finds the script in this folder, which is not on PATH.

TRY

Compare

Is the output the same as with bash show_args.sh? Why does ./ need the execute bit when bash does not?

Step 3: safe operands for wc

```
student@server: ~/oslab-work/lab7

$ wc -w -- 'input/one file.txt'
$ wc -w -- 'input/-dash.txt'
```

Expected counts: 2 and 1. -- ends option parsing for wc.

input/-dash.txt does not start with a dash, so it is not mistaken for an option. The checkpoint tests -dash.txt from inside input/, where it would be.

THINK

Observe

Why is a file name with a space one argument? What might for argument in $@ do differently?

PREDICT

Guess first, then try

Before you run the experiment. Ten minutes.

The prediction

PREDICTION

Write this down first

A script receives 'one file.txt' as one argument, but loops over an unquoted $@. What pieces might the loop see? Why might a file named -dash.txt cause a different kind of problem?

HOW

The rules

Alone, and without AI. On paper or in the course worksheet, before you run the command. You may compare with a neighbour for five minutes, but keep your own answer.

A wrong prediction costs nothing. It is marked for being made and for the evidence-based correction afterwards.

TASK 2

Build your personal file command

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1: look before you edit

student@server: ~/oslab-work/lab7

```
$ cat count_words.sh
$ cat 'input/one file.txt' 'input/-dash.txt'
```

Read the skeleton and the two input files first.

Both file names are in quotes, because one of them contains a space.

Step 2: what count_words.sh must do

Requirements

- No arguments: print a usage message to stderr, and exit with a non-zero status.
- For each regular file: print its word count and its original path on one labelled line.
- For a missing or non-regular input: print a useful message to stderr, carry on with the other arguments, and return non-zero overall.
- Keep argument boundaries, and pass file names safely. Do not use eval.

YOUR TURN

You write it

Build it from the guided argument loop. Different, reasonable output labels are accepted.

Step 3 and 4: grow it in small steps

student@server: ~/oslab-work/lab7

```
$ bash count_words.sh 'input/one file.txt'
$ bash count_words.sh 'input/one file.txt' 'input/-dash.txt'
```

Start with one valid file, then two. Check each before you add error handling.

To keep going after a failed input but still report it: set a variable to 0 at the start, change it when an input fails, and exit with it at the end.

Step 5: make it executable

student@server: ~/oslab-work/lab7

```
$ chmod u+x count_words.sh
$ ./count_words.sh 'input/one file.txt'
$ bash count_words.sh 'input/one file.txt'
```

Run it both ways and compare the behaviour.

DONE

Complete when

Both normal counts are right, the names stay intact, missing inputs are explained, and the final status tells whether any input failed.

Hints, one at a time

DEBUG

If you are stuck

Use show_args.sh to see how a name was split. That shows the problem at once.

Three hints

1. Quote each expanded path.
2. Display the arguments with the viewer when a name splits.
3. Test with [[-f "$path"]], and put -- before file name operands.

Normal, missing, leading dash, empty

Fifteen minutes, from minute 70 to 85.

Test 1 and 2: a normal run, and a failure

student@server: ~/oslab-work/lab7

```
$ bash count_words.sh 'input/one file.txt' 'input/-dash.txt' | tee evidence/normal.txt

$ {
$   bash count_words.sh missing.txt 'input/one file.txt'
$   printf 'status=%s\n' "$?"
$ } > evidence/edge.txt 2>&1
$ cat evidence/edge.txt
```

Save the normal run in evidence/normal.txt.

Expect: a message for the first input, a result for the second, and a non-zero status overall. Save it as evidence/edge.txt.

Test 3 and 4: a real leading dash, no input, an empty file

student@server: ~/oslab-work/lab7

```
$ cd input
$ bash ../count_words.sh '-dash.txt' 'one file.txt'
$ cd ..
```

Inside input/, the name -dash.txt really starts with a dash, so the script must put -- before it.

Also test no arguments, and an empty regular file of your own. An empty file has zero words. That is different from a missing file.

What each test shows

Case	Expected	A weak script that it exposes
Two valid files	Counts 2 and 1, names intact	A loop over unquoted $@
Missing, then valid	A message, a result, status above 0	A script that stops at the first error, or hides it
-dash.txt in input/	A count of 1	A script with no -- before the operands
No arguments	Usage on stderr, status above 0	A script that silently succeeds
An empty file	A count of 0, status 0	A script that treats empty as missing

Direct execution can fail with Permission denied: check the execute bit, or use bash script. See man wc and the Bash manual.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short question on paper

You answer one question on paper or on the course worksheet. Your earlier work does not need to be finished.

Practice question

The question on the next slide is public practice. A graded session may use a fresh private variant.

The question

CHECKPOINT

Practice question

The current folder holds a file literally named -dash.txt. Explain why wc -w -dash.txt may be misread, and give a safe command. Then explain why quoting alone does not end option parsing.

What to give

1. The result or diagnosis.
2. The mechanism.
3. One observation that could check it.

The instructor collects your answer before discussing the public key.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Correct your prediction

1. Keep your original prediction in view. Under it, write confirmed or corrected.
2. Cite the test that shows it.
3. Explain the OS mechanism in three to five sentences.

ANSWER

Three questions

(a) What does "$@" preserve? (b) Which problem does -- solve? (c) How does your script report one failed input while it processes the next?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab7/evidence"
$ cd "$OSLAB_WORKSPACE/lab7"
$ cp -- count_words.sh "$SUBMISSION_REPO/lab7/"
$ cp -- evidence/normal.txt evidence/edge.txt "$SUBMISSION_REPO/lab7/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the listed artifacts and two selected test records, not every terminal output.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab7
├── count_words.sh
├── evidence
│   ├── edge.txt
│   └── normal.txt
└── README.md

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Safe arguments, correct counts and a status for any failed input	3
Tests with a space, a leading dash, a missing file and an empty file	2
Quoting, option boundaries, execution mode and status explained; original prediction and correction	2
Individual checkpoint: result or diagnosis, and mechanism	2
Concise, attributable evidence and the required artifacts	1

Other valid commands and names earn credit. A naming difference costs points only when it breaks the result. If you use AI, note one useful suggestion and how you verified it.

Common mistakes

Mistake	What you see	Fix
for a in $@ without quotes	one file.txt split into two words	Write "$@"
No -- before operands	wc: invalid option	Put -- before the file names
Exit 0 after a failed input	A script that always looks successful	Keep a status variable, exit with it
Running ./script without chmod	Permission denied	chmod u+x, or use bash script

BEYOND THE CORE

PATH, login messages and the outbox

Optional extensions. Cross-user work needs a narrow folder the instructor prepared.

Lab 7

Your own commands on PATH

student@server: ~/oslab-work/lab7

```
$ mkdir -p ~/bin
$ cp count_words.sh ~/bin/count_words
$ export PATH="$HOME/bin:$PATH"
$ command -v count_words
/home/student/bin/count_words
```

PATH is the list of folders the shell searches for a command name. Add your own folder in front, and the script runs without ./.

export PATH=... lasts for this shell only. Put it in ~/.bashrc to keep it, and read that file again with source ~/.bashrc.

Login messages, outbox and the mailbox idea

LOGIN

A message when you log in

A line in ~/.bash_profile or ~/.bashrc can print a notice. Preview it with a plain command before you rely on it.

OUTBOX

An owned outbox

A folder you own where a script leaves messages. Only you write to it, so permissions stay simple.

MAILBOX

A cross-user mailbox

Needs a narrow shared folder the instructor prepared, with write permission only for the people who need it. Never open your own home.

A program that runs with its owner's rights (setuid) is a concept for a prepared virtual machine. It is not part of this lab.

Key takeaways

01

A script is saved commands

A shebang names the interpreter, and chmod u+x lets you run the file directly.

02

Quote what you expand

"$@" keeps each argument whole. Unquoted, a space splits it into two words.

03

-- ends the options

Use it before file names, so a leading dash cannot be read as an option.

04

The status is the message

Exit non-zero when any input failed, even if you carried on with the rest.

Next session

COMING UP

Lab 8: secure scripts, races and file locking

Two scripts touch one file at once. See what goes wrong, and fix it with a lock.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 2, Operating-System Structures (the command interpreter), and the Bash manual.